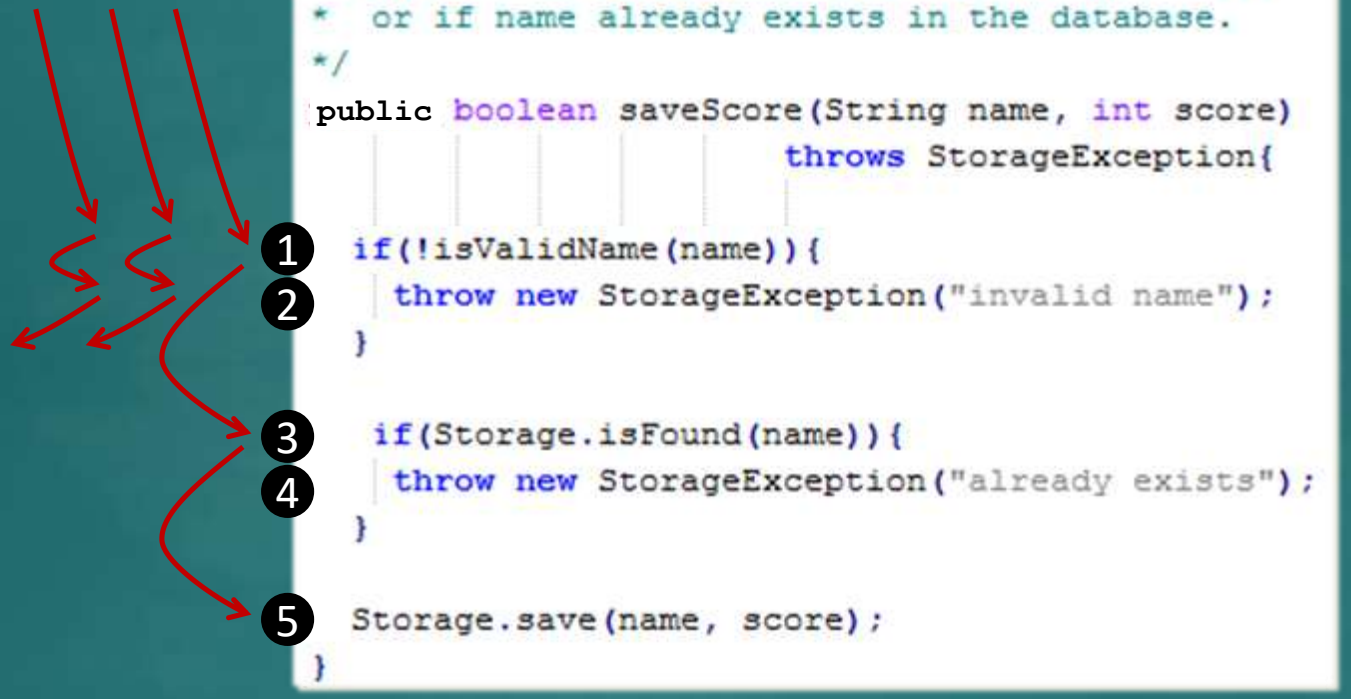
a) Estimate the statement coverage of this method for these three test cases.



```
/**
* Throws StorageException if name is  not valid
*  or if name already exists in the database.
*/
public boolean saveScore(String name, int score)
                                throws StorageException{

1   if(!isValidName(name)){
2     throw new StorageException("invalid name");
    }

3    if(Storage.isFound(name)){
4     throw new StorageException("already exists");
    }

5   Storage.save(name, score);
}
```

Test input	Expected
"new guy", 0	success
null , -3	Name invalid -> exception
"", 5	Name invalid -> exception

i. 0%

ii. about 50%

iii. about 80%

iv. 100%

b) If all tests of a software pass, and those tests achieve 100% path coverage, (all possible execution paths through the code have been tested), can the software still have bugs?

Q1

Estimate the statement coverage of this method for these three test cases.

```
/**
* Throws StorageException if name is  not valid
*  or if name already exists in the database.
*/
public boolean saveScore(String name, int score)
                                throws StorageException{

1   if(!isValidName(name)){
2     throw new StorageException("invalid name");
    }

3    if(Storage.isFound(name)){
4     throw new StorageException("already exists");
    }

5   Storage.save(name, score);
}
```

Test input	Expected
"new guy", 0	success
null , -3	Name invalid -> exception
"", 5	Name invalid -> exception

i. 0%

ii. about 50%

iii. about 80%

iv. 100%

Measuring coverage in this case makes us wonder,

* Should we add more test cases, cover statement 4?
* Should we reconsider the 3rd test case?

Q2

If all tests pass and those tests achieve 100% path coverage
(i.e., all possible execution paths through the code have been tested),
can the software still have bugs?

( ) Yes

A bug can also be the result of a 'missing path' in the code.

High coverage alone cannot guarantee bug free code.

( ) No

Suppose this is the answer. Then, should we aim for 100% path coverage?

A non-trivial system is likely to have a HUGE number of distinct paths.

Q3

What to consider when deciding if an association is a composition?

- [ ] If `Foo` objects act as containers for `Bar` objects, it is composition.
- [ ] As long as `Bar` objects get deleted when `Foo` objects are deleted, it is composition.
- [ ] Concepts represented by the two classes form a *whole-part* relationship.
- [ ] Not always possible to judge from the code alone.

In the exam, there can be additional clues in the description/comments
e.g., `// a Foo is composed of Bars`

This course follows a narrower view of composition, to avoid overuse.
Composition tells us 'this cluster of objects is actually one entity'.
Not a strict UML rule.
Others may differ. When in Rome, do as the Romans do.

What if we make a mistake on this in a real project?

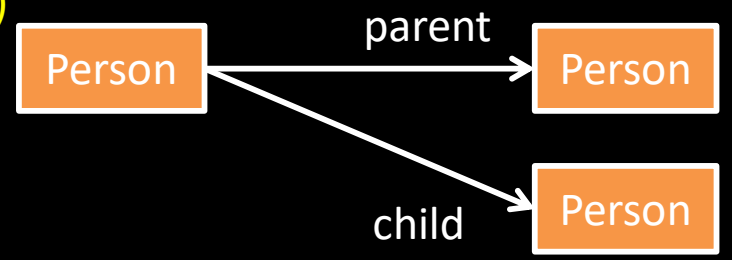
Q4

Context: A software to manage a competition. Contestants are parent-child pairs.
Which class diagram is the best match to the code?

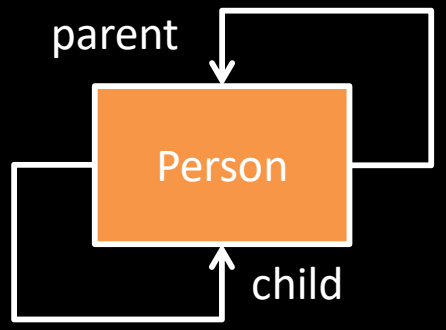
```
class Person {
    Person parent;
    Person child;
    // …
}
```

The diagram should show the *intent* of the design, not necessarily a literal translation.

(a)

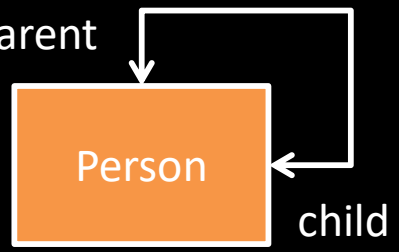


(b)



2 *unidirectional* associations

(c)



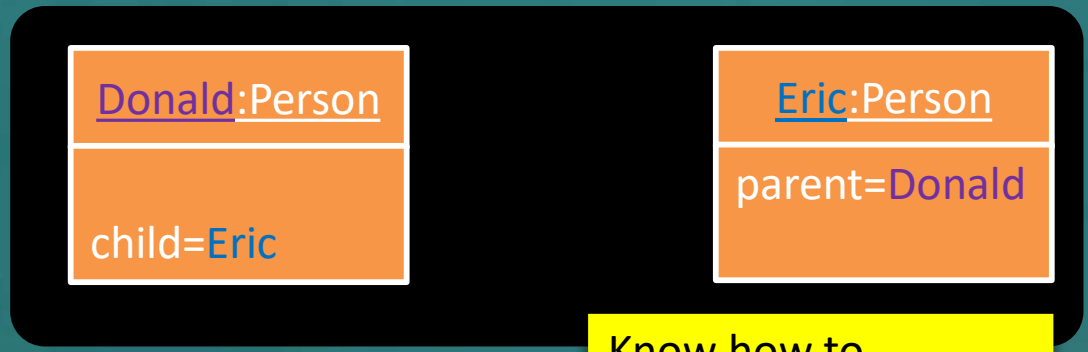
a *bidirectional* association

Is the relationship mutual? i.e., if one `Person` object is the `parent` of another, does that make the second one the `child` of the first, and vice versa?

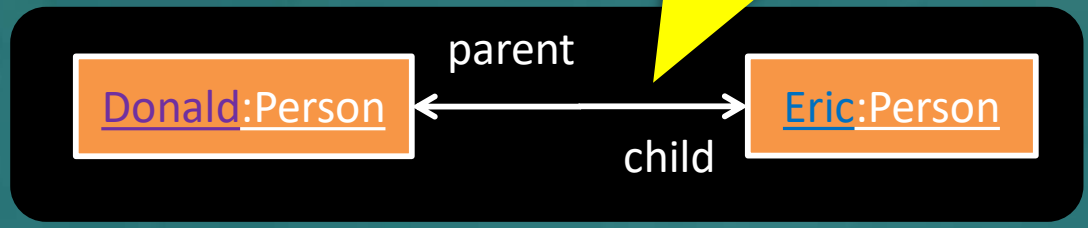
Let’s dig in a bit deeper to see how this design works in practice …

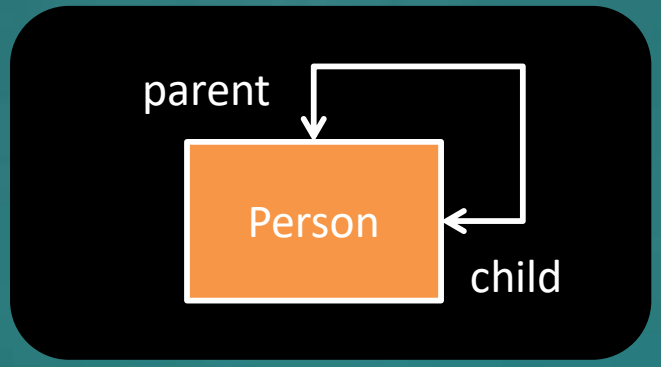
```
class Person {
    Person parent;
    Person child;
    // …
}
```

Just one parent only?



Know how to recognize this type of mutual relationships



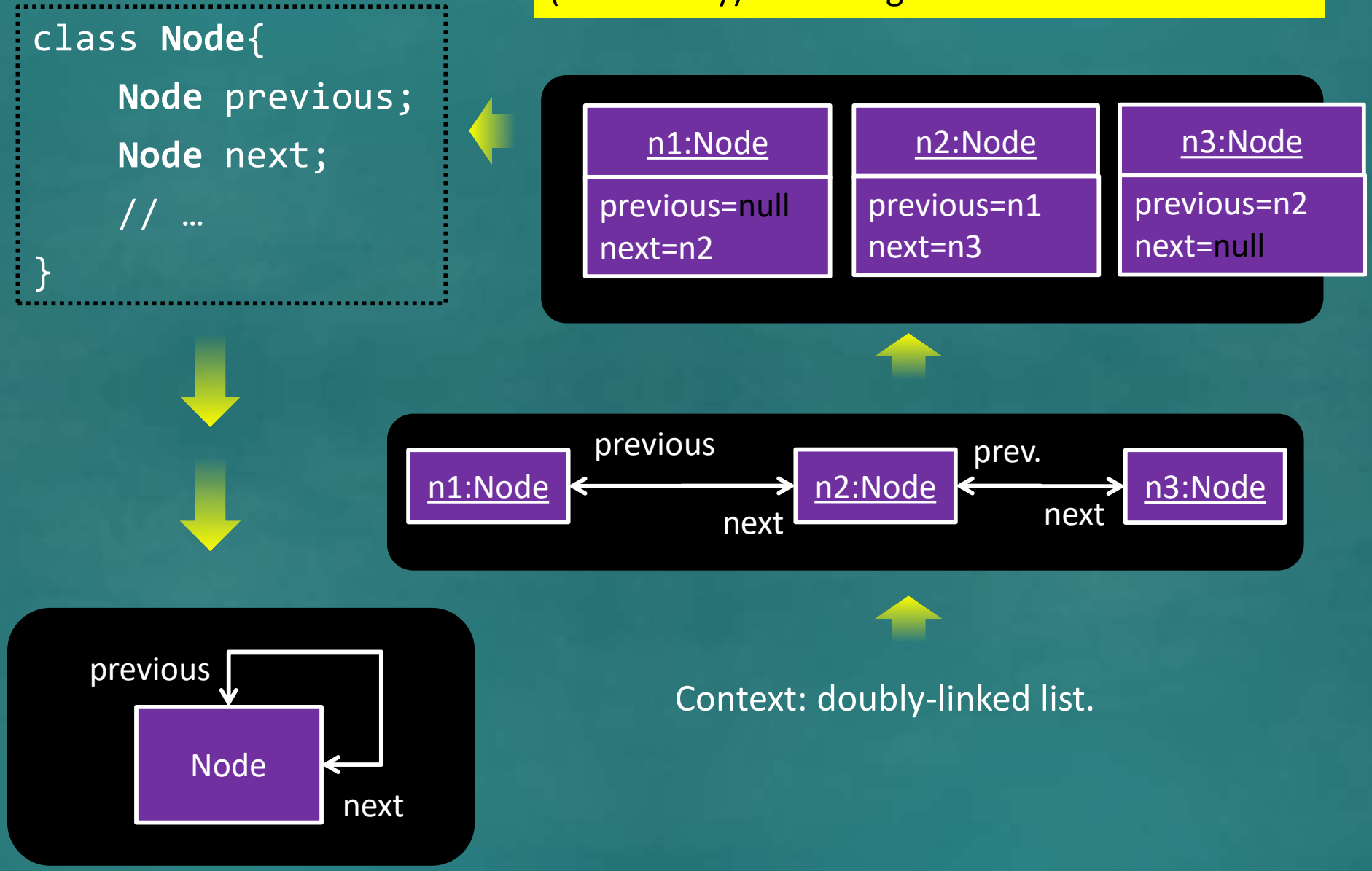


Note how the design is influenced by the context

Context: A software to manage a competition.
Contestants are parent-child pairs.

Note how object diagrams can help figure out (or to verify) class diagrams

```
class Node{
    Node previous;
    Node next;
    // …
}
```

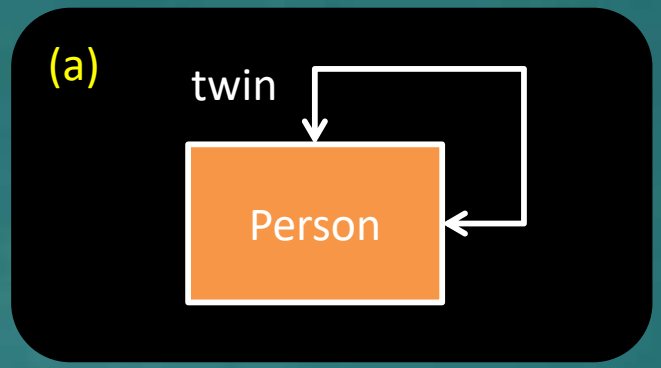


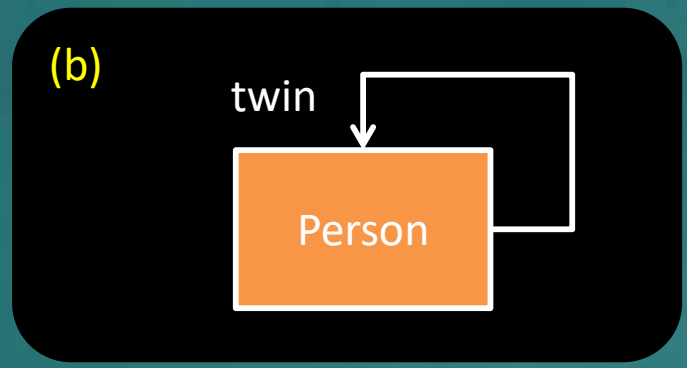
Context: doubly-linked list.

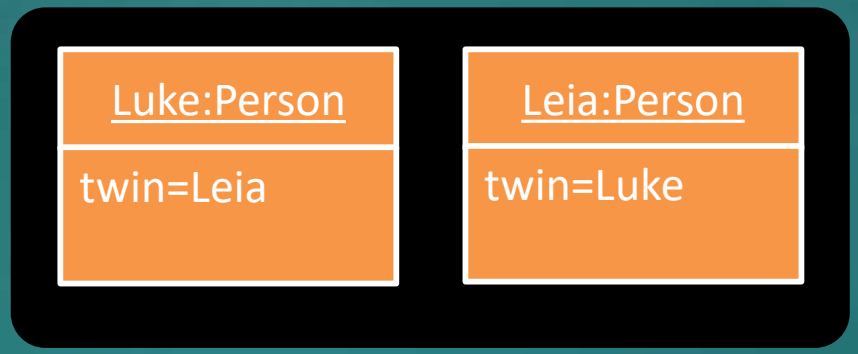
Which class diagram is the best match for the code?

```
class Person {
    Person twin;
    // …
}
```

A bi-directional association can be implemented using only one variable too (if both play the same role)



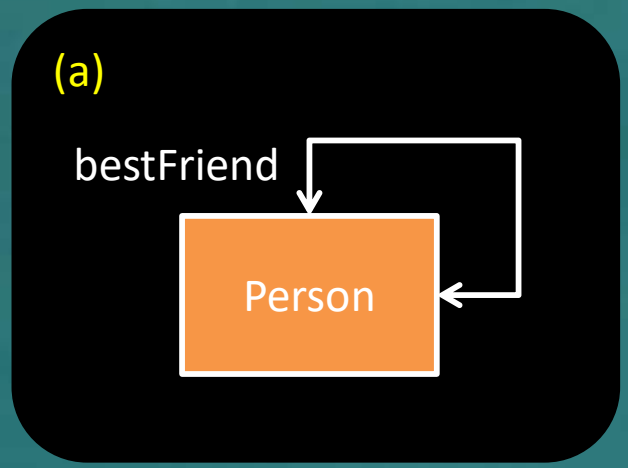


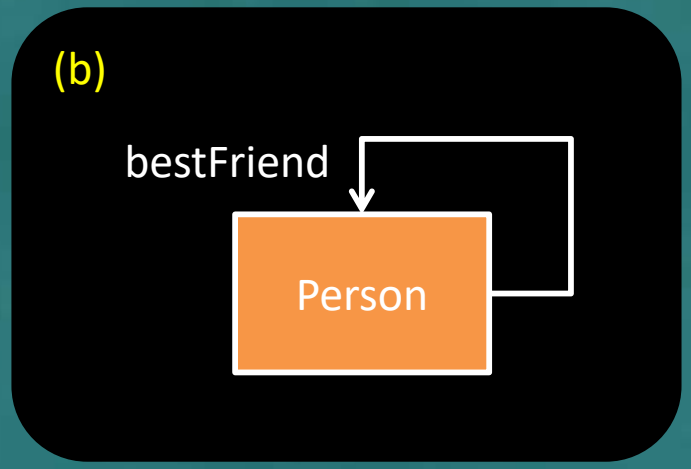


Q

Which class diagram is the best match for the code?

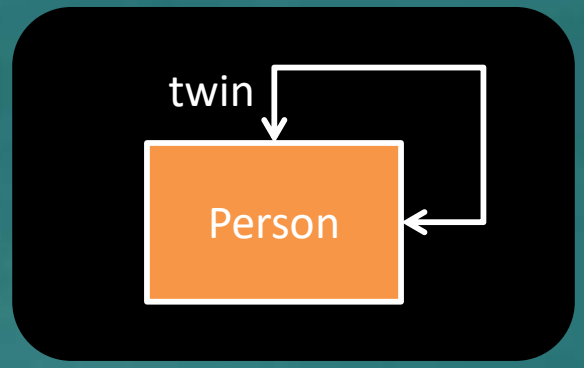
```
class Person {
    Person bestFriend;
    // …
}
```

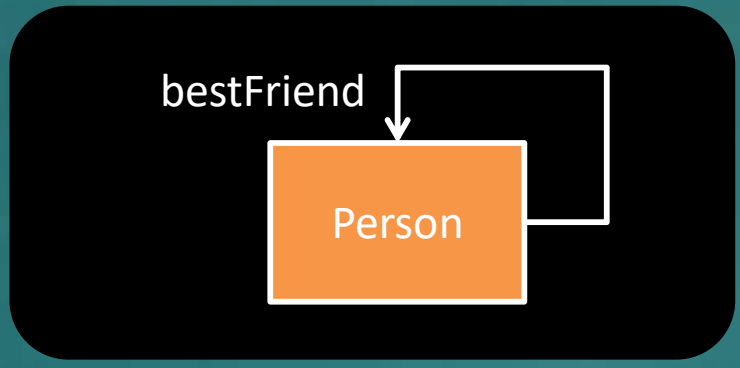




```
class Person {
    Person twin;
    // …
}
```

```
class Person {
    Person bestFriend;
    // …
}
```





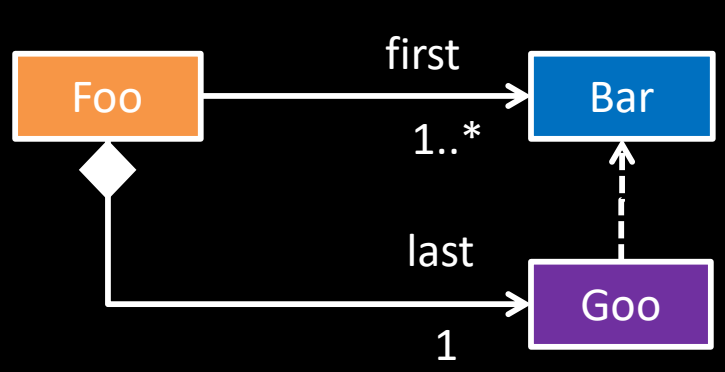
Reminders:

- Just the code may not be enough to draw the matching UML diagram.
- Auto-generated UML diagrams may not be ideal.

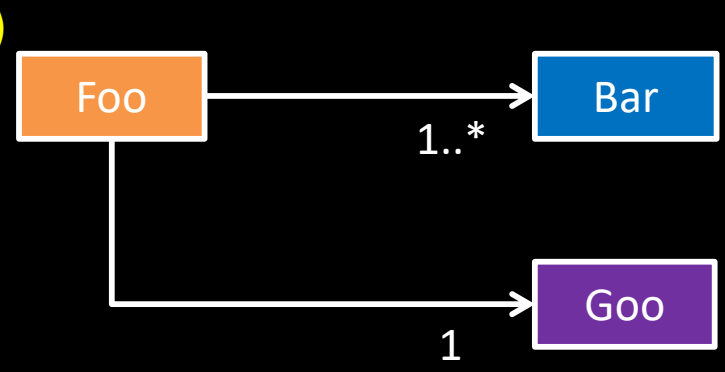
Q

Which class diagram is best? All correct, all about the same design, just different levels of details

(a)



(b)



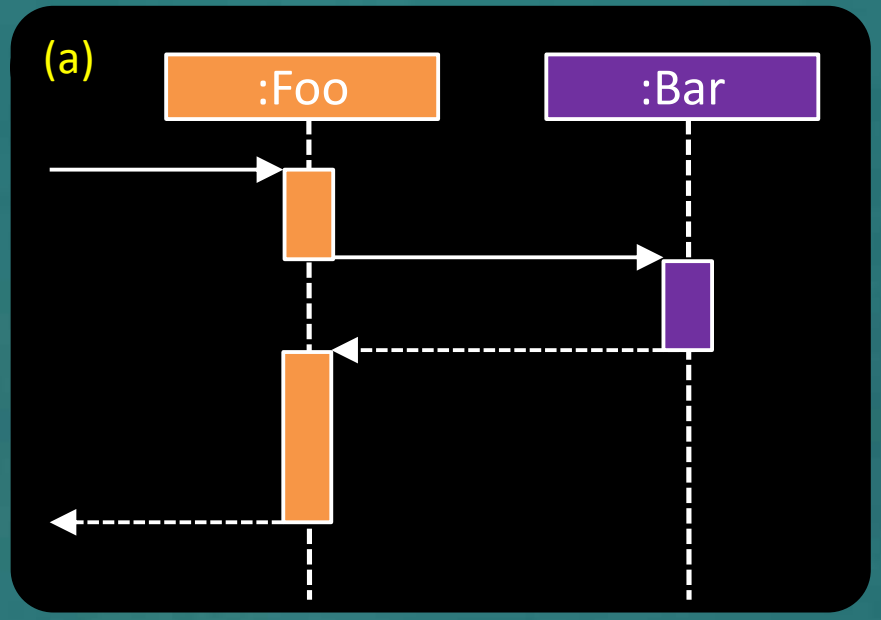
(c)

(d) It depends on the purpose of the diagram

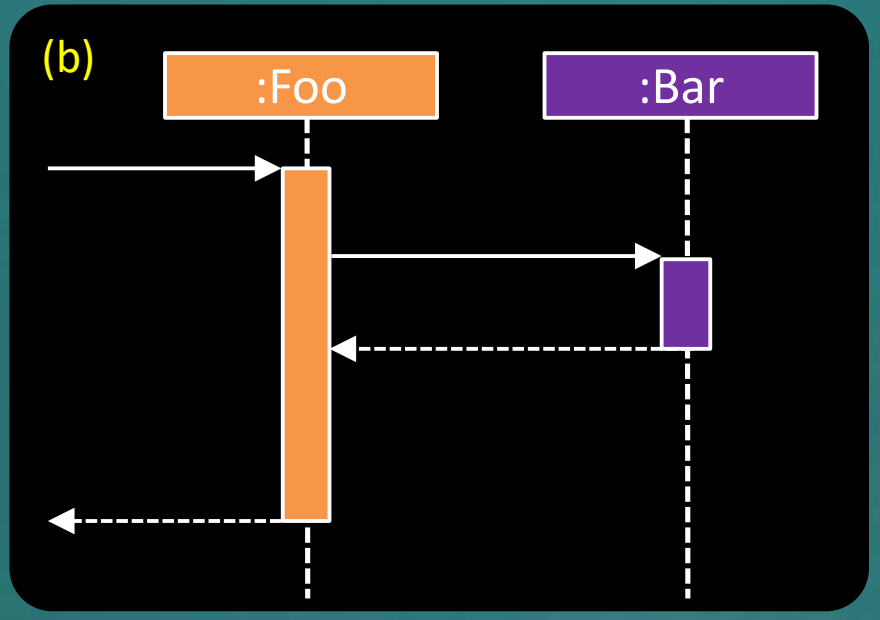
Q

Which sequence diagram is wrong?

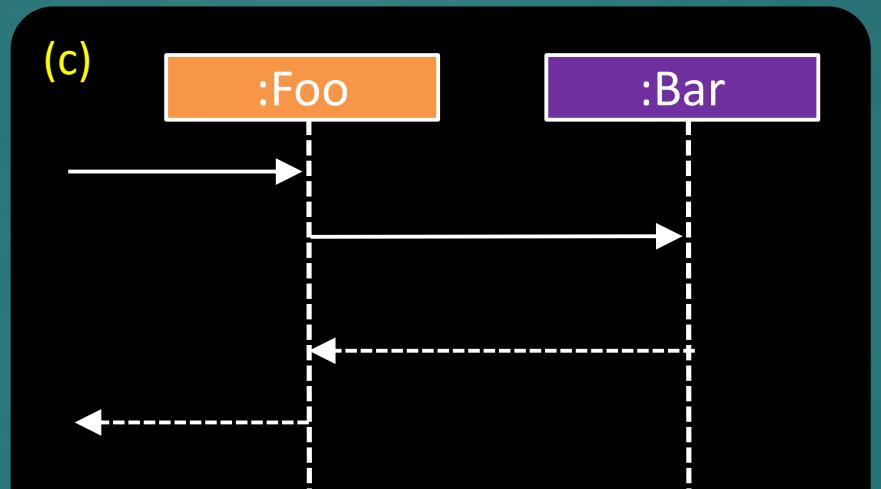
(a)



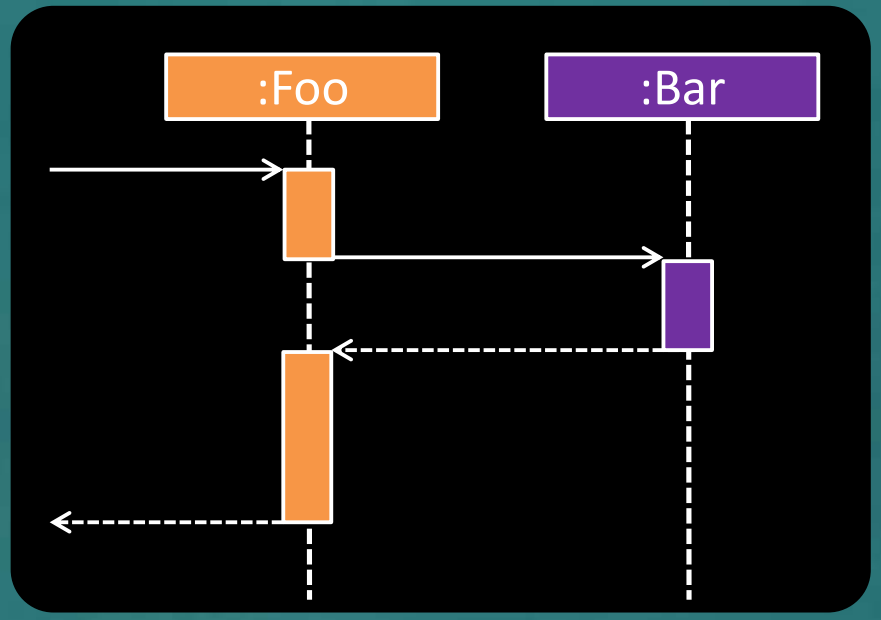
(b)



(c)



(d) None. All are correct



FYI: This kind of non-continuous activation bars are used to show asynchronous method calls (i.e., the caller doesn't block until the called method returns).

But not in scope of this course.

A statement in the code can be *covered* by these:

- ❑ Unit testing
- ❑ Integration testing
- ❑ System testing
- ❑ Smoke testing
- ❑ Acceptance testing
- ❑ Alpha/beta testing
- ❑ Regression testing
- ❑ TDD

Can be done in hybrid mode (i.e., combined)

Why not just do this one?

- Too late
- Too costly
- Too slow
- Doesn’t test some things

Q

Dependency injection is most related to:

- Unit testing
- Integration testing
- System testing

Preview of next week

Week 9

Topics:

- [W9.1] **Conceptual Class Diagrams (aka OODMs)** ⚡⚡⚡
- [W9.2] **Activity Diagrams** ⚡⚡⚡
- [W9.3] **Concept**
- [W9.4] **Archite**
- [W9.5] **Design**
- [W9.6] **[Revisiting] SDLC Process Models** ⚡⚡
- [W9.7] **SDLC Process Models (continued)** ⚡⚡⚡
- [W9.8] **Writing Developer Documents** ⚡⚡

Covered in quiz part 1 (immediate answers). Used for the next week's tutorial

Admin:

1. Submit weekly quiz Ⓟ

tP: v1.3

1. Plan the first product release (`v1.3`)
2. Manage the iteration, and deliver `v1.3`

Thu, Mar 20th 23:59

Week 9

Topics:

- [W9.1] **Conceptual Class Diagrams (aka OODMs)** ⚡⚡⚡
- [W9.2] **Activity Diagrams** ⚡⚡⚡
- [W9.3] **Conceptualizing a Design** ⚡⚡
- [W
- [W
- [W
- [W
- [W

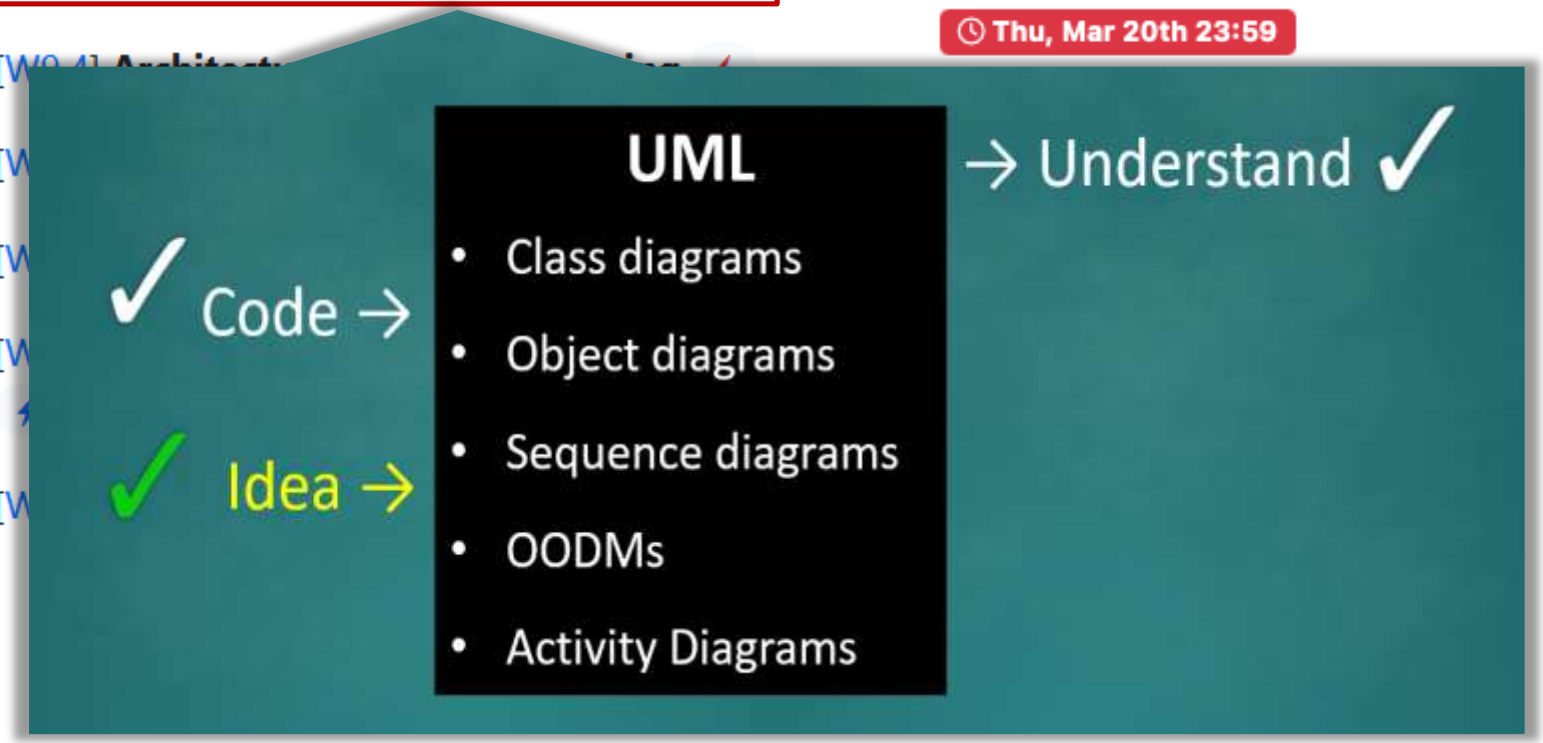
Admin:

1. Submit weekly quiz (P)

tP: v1.3

1. Plan the first product release (`v1.3`)
2. Manage the iteration, and deliver `v1.3`

Thu, Mar 20th 23:59



Week 9

Topics:

- [W9.1] **Conceptual Class Diagrams (aka OODMs)** ⚡⚡⚡
- [W9.2] **Activity Diagrams** ⚡⚡⚡
- [W9.3] **Conceptualizing a Design** ⚡⚡
- [W9.4] **Architecture Diagrams: Drawing** ⚡
- [W9.5] **Design Pri**
- [W9
- [W9
  ⚡⚡
- [W9

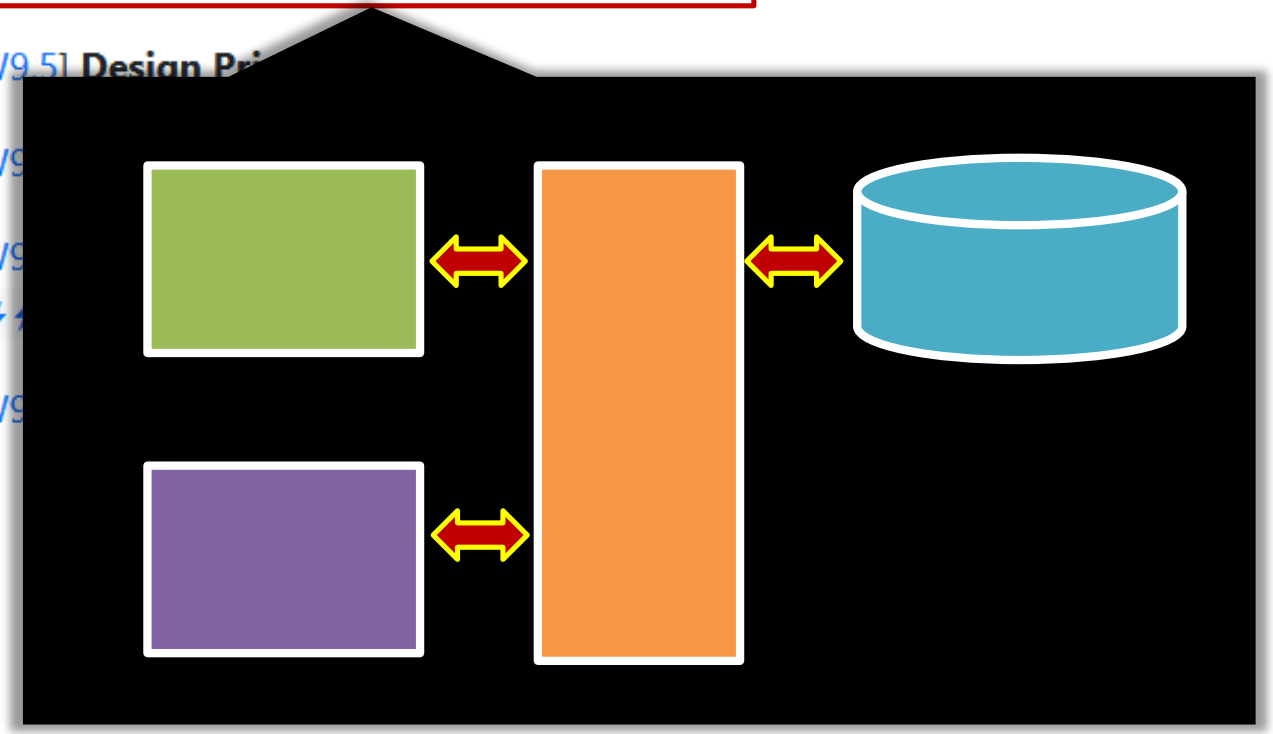
Admin:

1. Submit weekly quiz Ⓟ

tP: v1.3

1. Plan the first product release (`v1.3`)
2. Manage the iteration, and deliver `v1.3`

Thu, Mar 20th 23:59



experience

… is what you get
when you didn’t get what you wanted.

experience

Internet/Intranet Developer:

BS degree required or work equivalent. Must have HTML, XTL, Java, .NET and VB.NET. Preferred experience with MS ISS, E-Commerce, MS-SQL, Dot Net Nuke, and Flash/Dreamweaver. Salary commensurate with experience.

Project Lead:

BS degree required. Minimum of 4 years programming experience, 2 years with .Net. Develops and maintains systems, investigates current systems and oversees development of new systems. Salary commensurate with experience.

Systems Architect:

BS degree required. Minimum of 5 years experience in architect enterprise solutions. Designing software architecture, extremely technically focused position. .NET, UML, XML, algorithms. Salary commensurate with experience.

… is valuable.

Learning from experience

… is too costly.

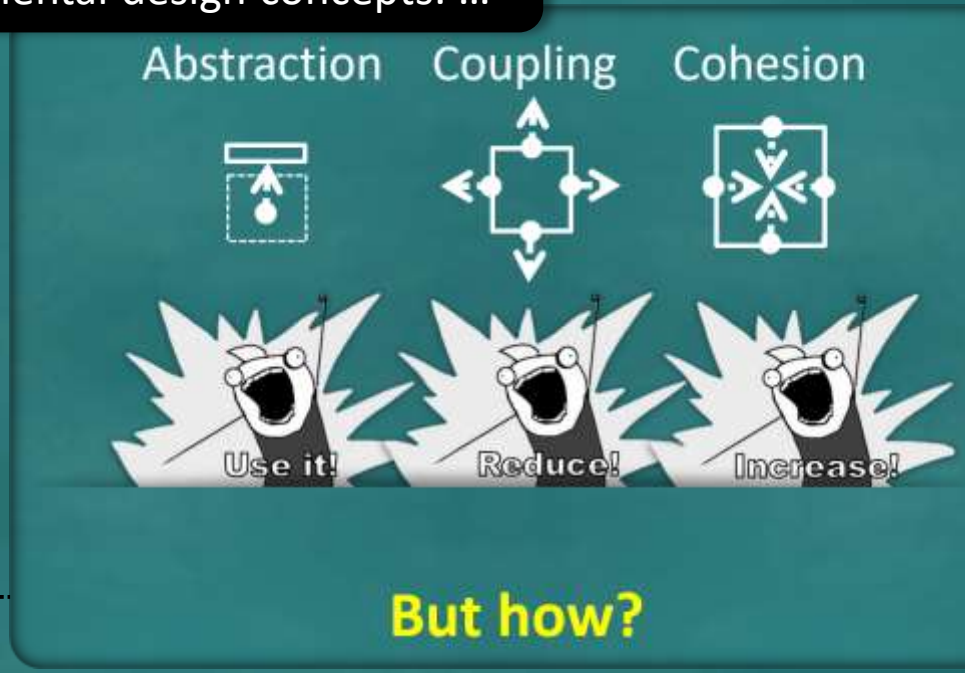
Learning from **experience**

Note to self: never volunteer to be the headstand guy

Gems of Wisdom:
Software Engineering Principles.

Design Principles

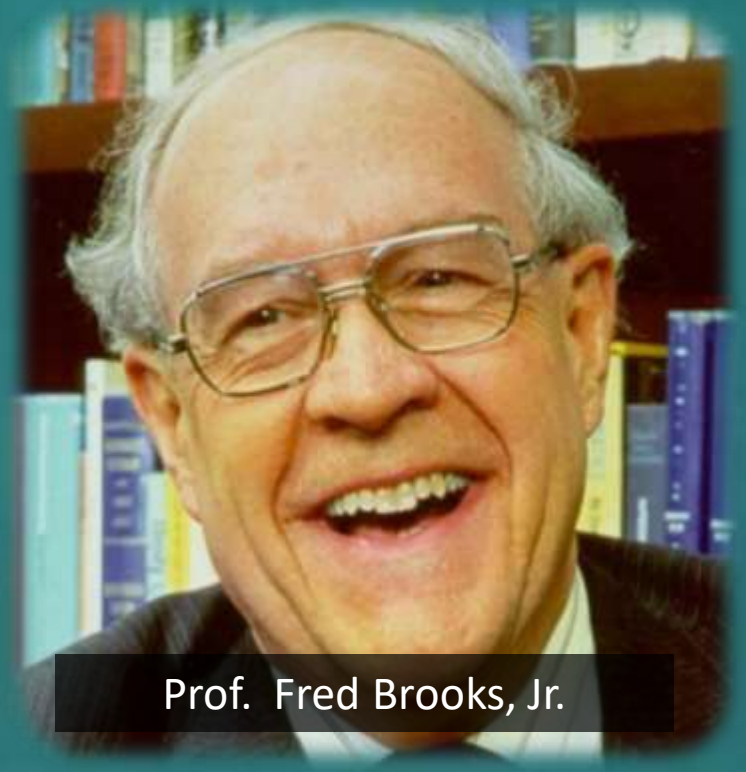
These build on top of the three fundamental design concepts: …



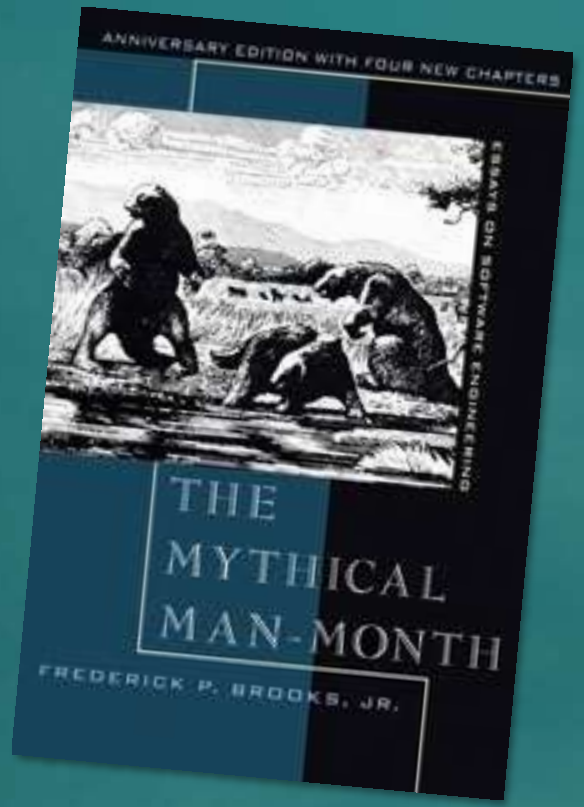
- ☑ **SoC**: Separation of Concerns
- ☑ **LoD**: Law of Demeter
- ☐ **SOLID** Principles
  - ☑ SRP – Single Responsibility Principle
  - ☑ OCP – Open-Closed Principle
  - ☑ LSP – Liskov Substitution Problem
  - ☐ ISP – Interface Segregation Principle
  - ☐ DIP – Dependency Inversion Principle
- ☐ DRY: Don't Repeat Yourself
- ☐ YAGNI: You Aren't Gonna Need It!

There are other types of principles too …


IBM
SYSTEM/360



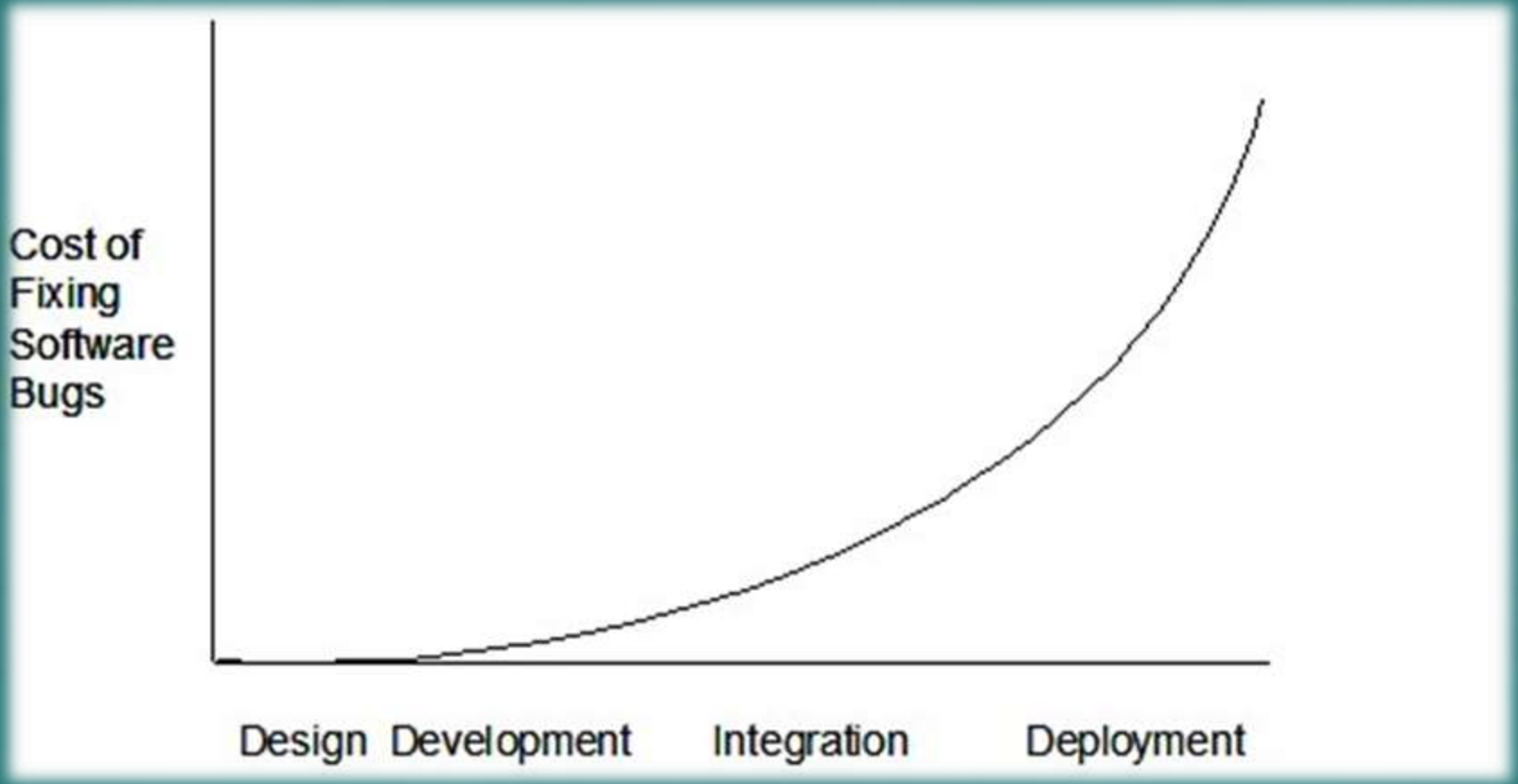
Prof. Fred Brooks, Jr.



Brooks's law :

- Adding manpower to a late software project makes it later.
- Plan to throw one (implementation) away; you will, anyhow.
- The second system is the most dangerous system that an architect ever designs.
- The bearing of a child takes nine months, no matter how many women are assigned. Many software tasks have this characteristic too ...

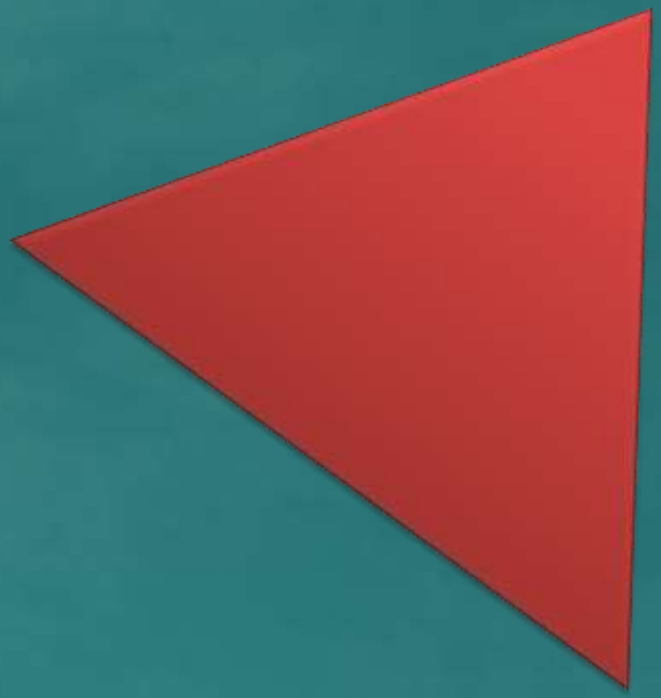
The later you find a bug, the more it costs



Good, cheap, fast: select any two

Good

Cheap



Fast

“Principles” vary in rigor, depth, credibility …

"Some principles are nicely captured in famous quotes"

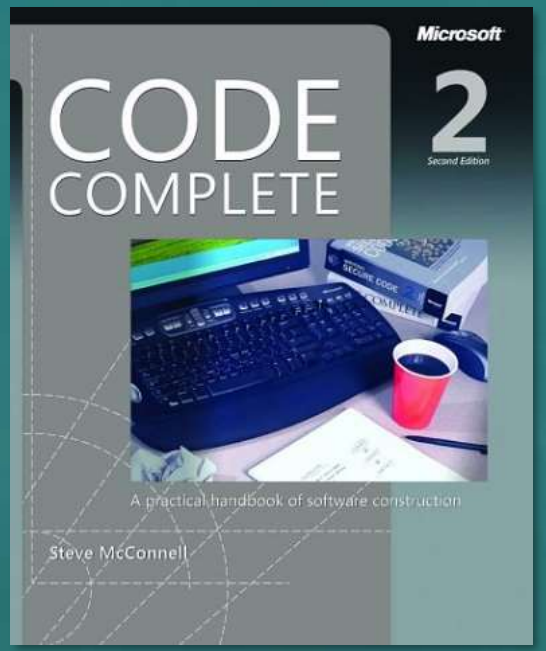
-- *me*

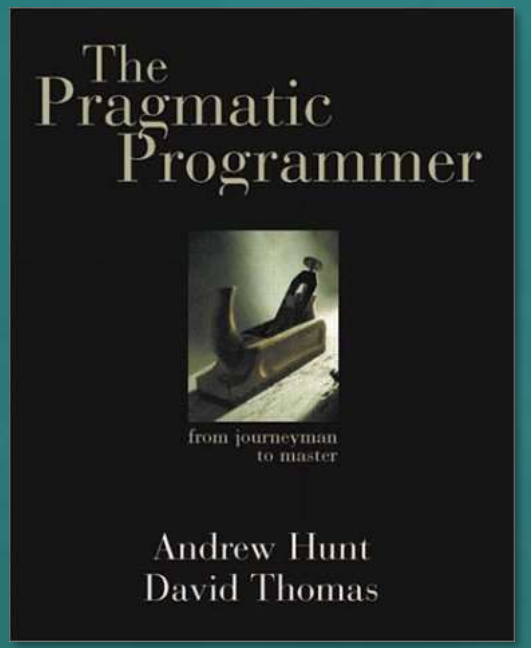
E.g.

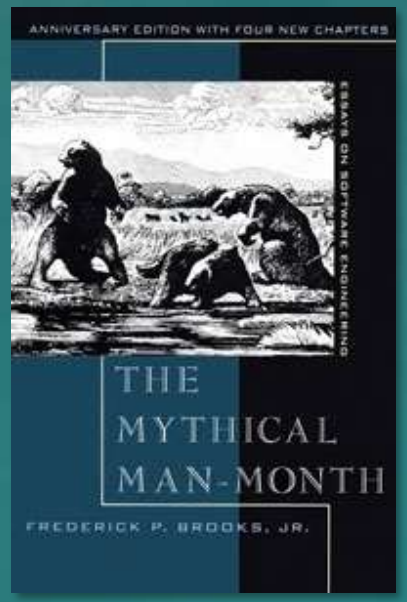
The first 90% of the code accounts for the first 90% of the development time...The remaining 10% of the code accounts for the other 90% of the development time. -- *Tom Cargill*

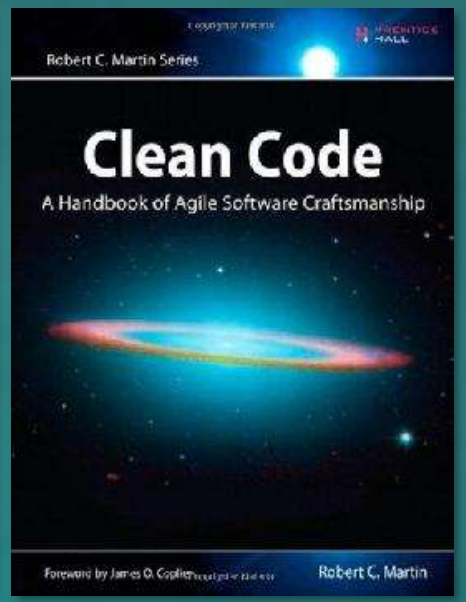
More SE quotes https://www.comp.nus.edu.sg/~damithch/pages/SE-quotes.htm

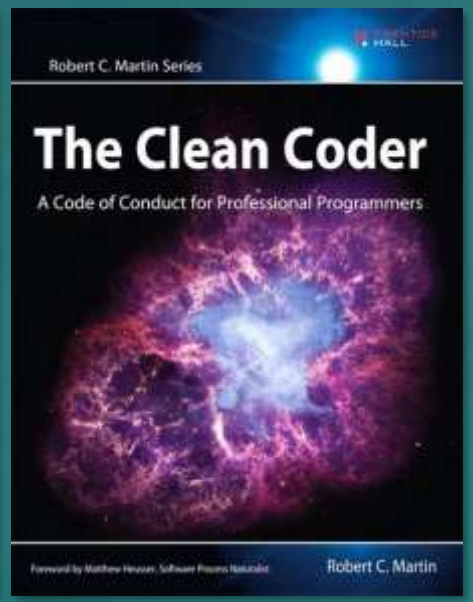
Which of these is NOT a book containing practical SE ‘wisdom’?











Week 9

Topics:

- [W9.1] **Conceptual Class Diagrams (aka OODMs)** ⚡⚡⚡
- [W9.2] **Activity Diagrams** ⚡⚡⚡
- [W9.3] **Conceptualizing a Design** ⚡⚡
- [W9.4] **Architecture Diagrams: Drawing** ⚡
- [W9.5] **Design Principles** ⚡⚡
- [W9.6] **[Revisiting] SDLC Process Models** ⚡⚡
- [W9.7] **SDLC Process Models (continued)** ⚡⚡⚡
- [W9.8] **Writing Developer Documents** ⚡⚡

Admin:

1. Submit weekly quiz (P)

tP: v1.3

1. Plan the first product release (`v1.3`)
2. Manage the iteration, and deliver `v1.3`

Thu, Mar 20th 23:59

Week 9

Topics:

- [W9.1] **Conceptual Class Diagrams (aka OODMs)** ⚡⚡⚡
- [W9.2] **Activity Diagrams** ⚡⚡⚡
- [W9.3] **Conceptualizing a Design** ⚡⚡
- [W9.4] **Architecture Diagrams: Drawing** ⚡
- [W9.5] **Design Principles** ⚡⚡
- [W9.6] **[Revisiting] SDLC Process Models** ⚡⚡
- [W9.7] **SDLC Process Models (continued)** ⚡⚡⚡
- [W9.8] **Writing Developer Documents** ⚡⚡

Admin:

1. Submit weekly quiz (P)

tP: v1.3

1. Plan the first product release (`v1.3`)
2. Manage the iteration, and deliver `v1.3`

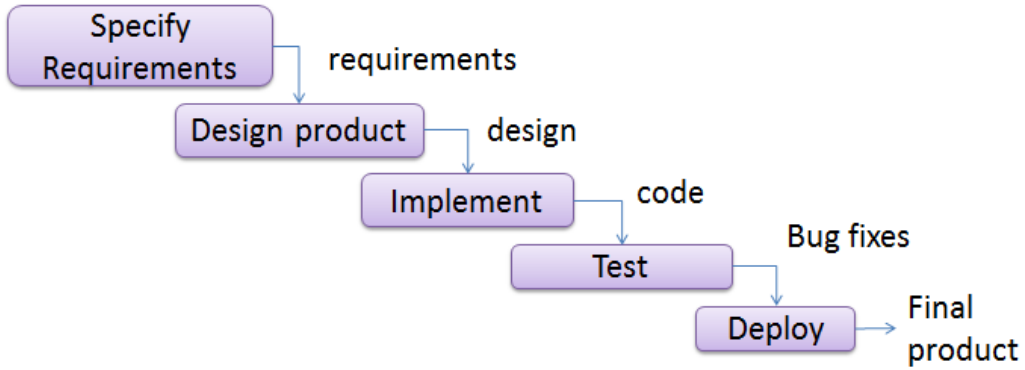
Thu, Mar 20th 23:59

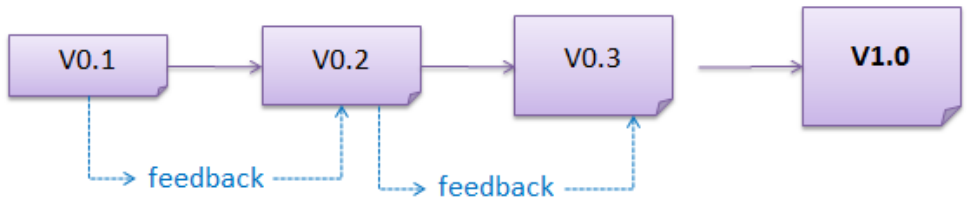
Building blocks

Sequential
[aka waterfall]

Iterative
(breadth-first)

Iterative
(depth-first)





Sequential
Iterative
breadth-first
depth-first

Actual process models

Scrum!

Sequential
Iterative
breadth-first
depth-first

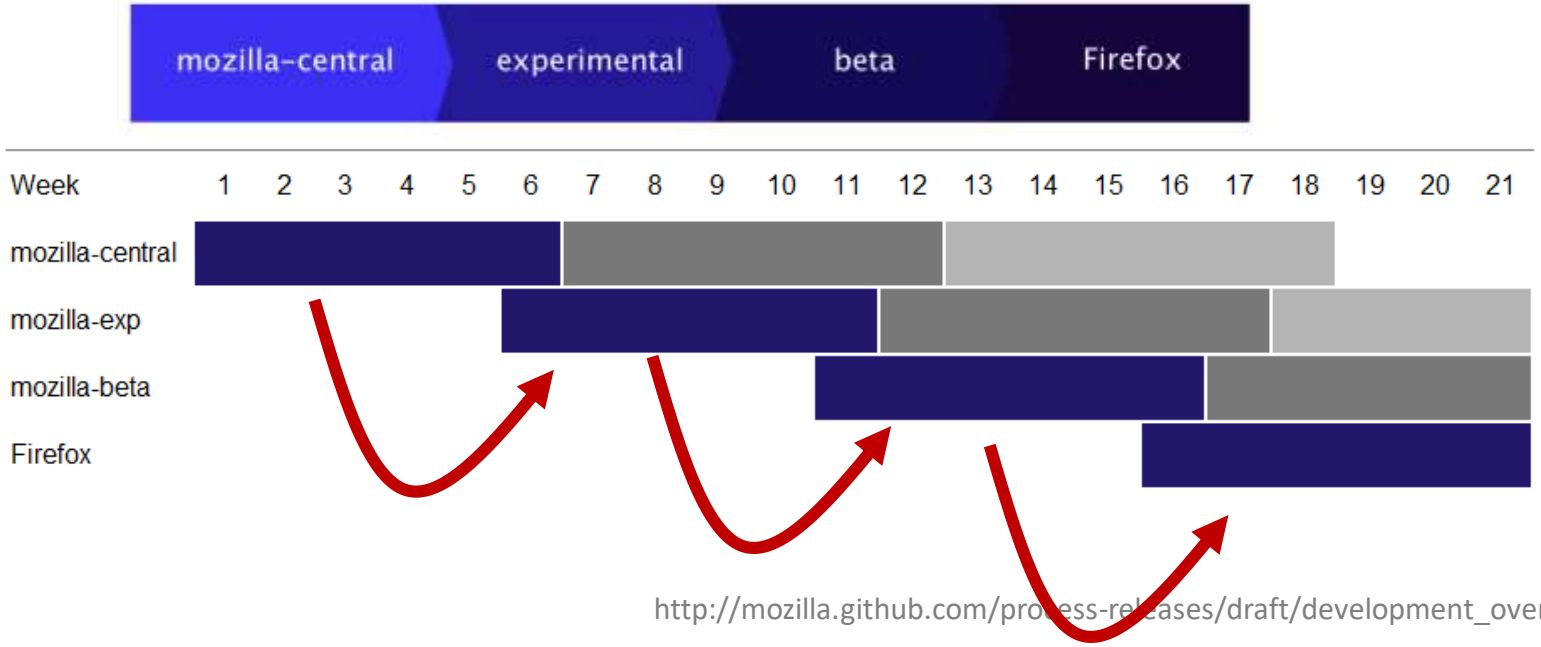
Actual process models

Scrum!

In the video only

Some example processes used in well-known companies …

All changes to Firefox source code are initially integrated in the mozilla-central Mercurial repository. At scheduled intervals, the changes are imported from mozilla-central to release channels with wider audiences, such as the beta channel. Eventually, the changes appear in the general Firefox



http://mozilla.github.com/process-releases/draft/development_overview/

Week 9

Topics:

- [W9.1] **Conceptual Class Diagrams (aka OODMs)** ⚡⚡⚡
- [W9.2] **Activity Diagrams** ⚡⚡⚡
- [W9.3] **Conceptualizing a Design** ⚡⚡
- [W9.4] **Architecture Diagrams: Drawing** ⚡
- [W9.5] **Design Principles** ⚡⚡
- [W9.6] **[Revisiting] SDLC Process Models** ⚡⚡
- [W9.7] **SDLC Process Models (continued)** ⚡⚡⚡
- [W9.8] **Writing Developer Documents** ⚡⚡

Admin:

1. Submit weekly quiz (P)

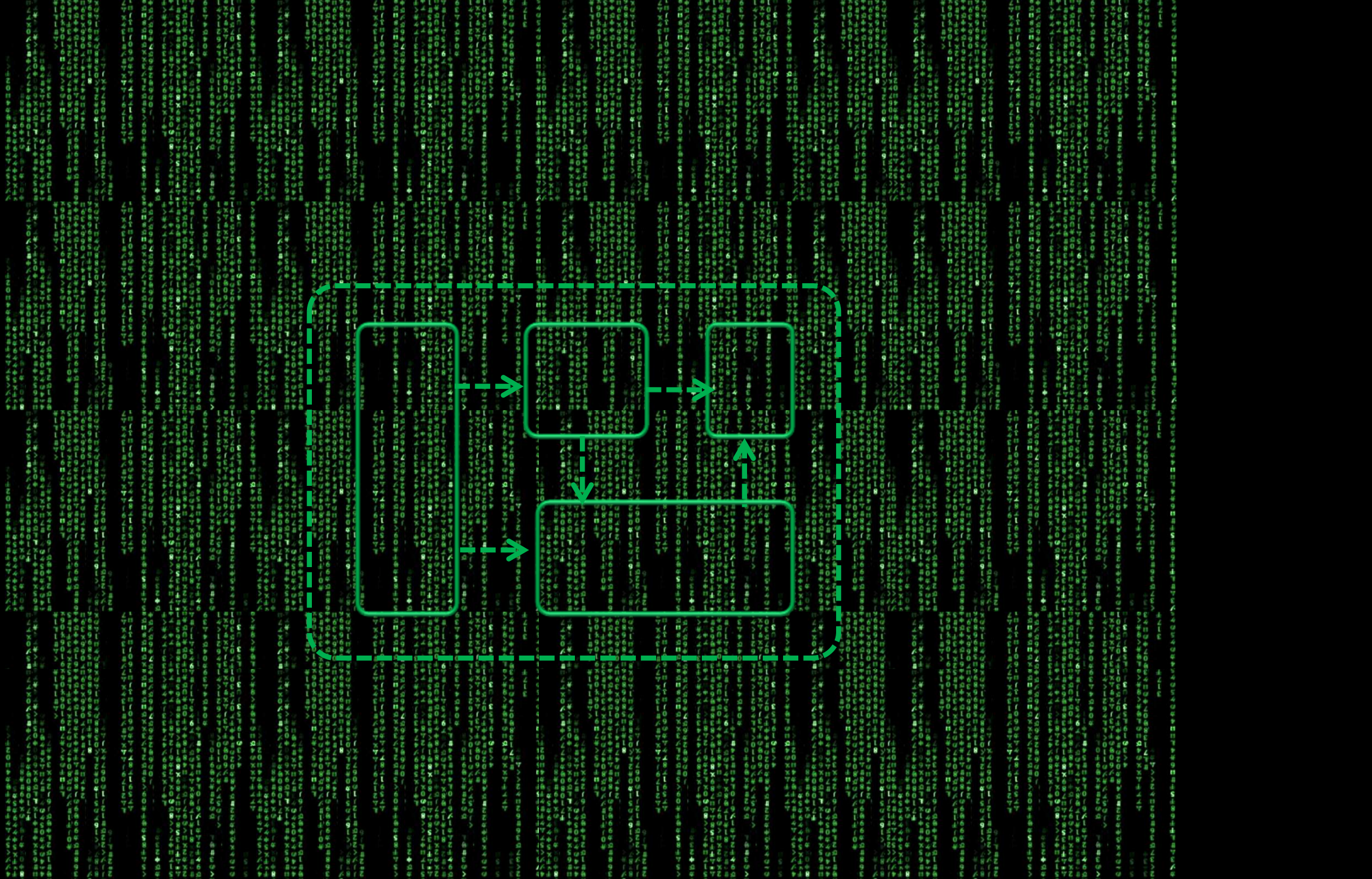
tP: v1.3

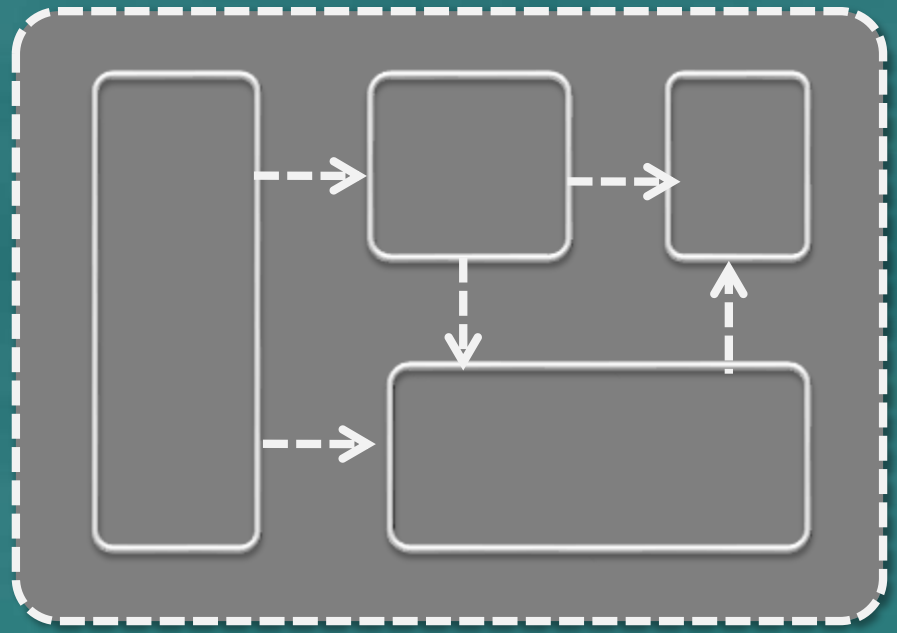
1. Plan the first product release (`v1.3`)
2. Manage the iteration, and deliver `v1.3`

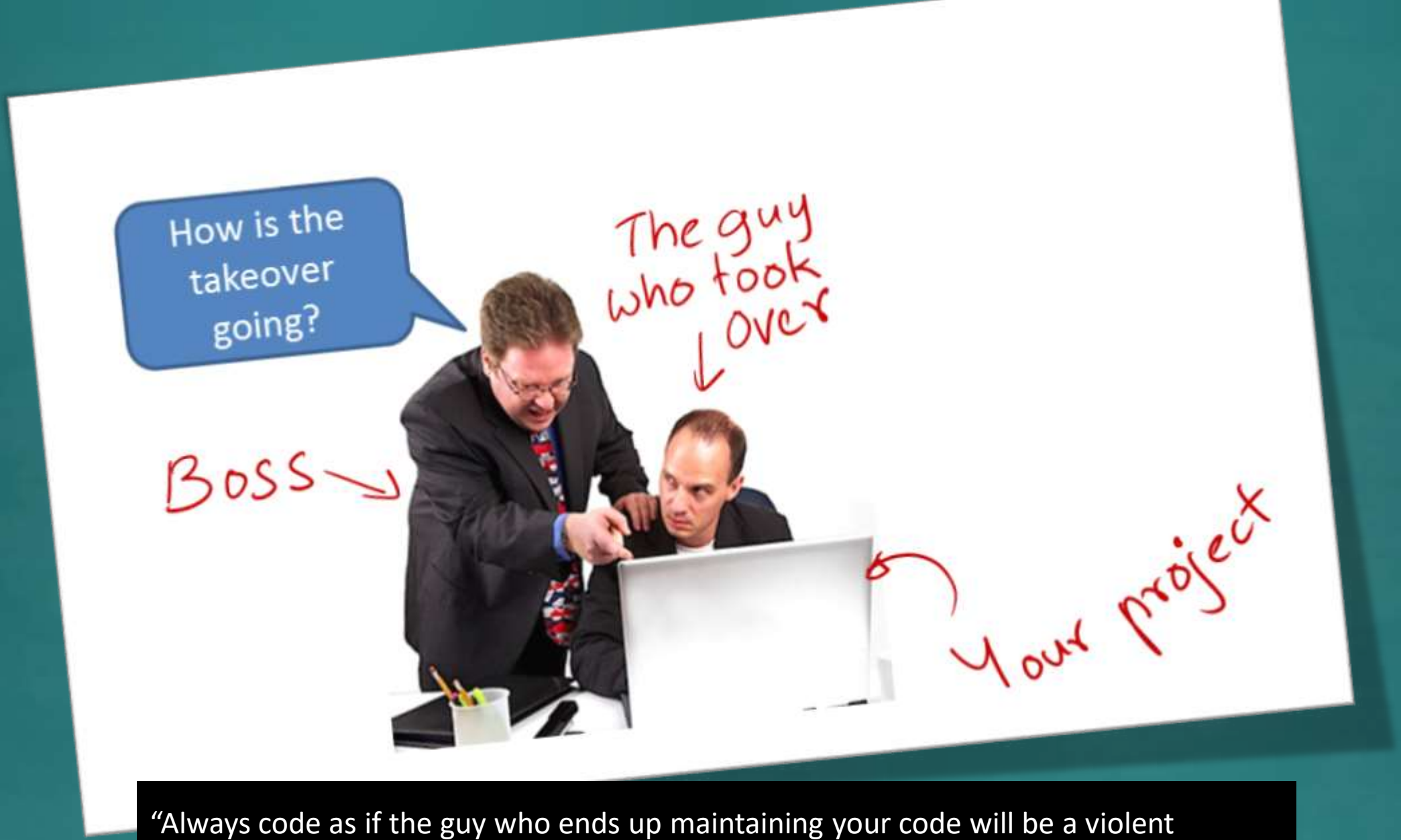
Thu, Mar 20th 23:59

```
# Generic relations were moved in Django revision 5172
try:
    from django.contrib.contenttypes import generic
except ImportError:
    import django.db.models as generic

class Tag(models.Model):
    """
    A basic tag. <code> RULES!!!</code>
    """
    name = models.CharField(maxlength=50, unique=True,
            db_index=True, validator_list=[isTag])

    objects = TagManager()

    class Meta:
        db_table = 'tag'
        verbose_name = 'Tag'
```

```
# Generic relations were moved in Django revision 5172
try:
    from django.contrib.contenttypes import generic
except ImportError:
    import django.db.models as generic

class Tag(models.Model):
    """
    A basic tag. <code> RULES!!!</code>
    """
    name = models.CharField(maxlength=50, unique=True,
            db_index=True, validator_list=[isTag])

    objects = TagManager()

    class Meta:
        db_table = 'tag'
        verbose_name = 'Tag'
```







"Always code as if the guy who ends up maintaining your code will be a violent psychopath who knows where you live"

Q

Developer documentation,

- ❑ a necessary evil
- ❑ should be done with maintainability in mind
- ❑ should be minimal yet sufficient
- ❑ should include volatile/low-level details

Week 9

Topics:

- [W9.1] **Conceptual Class Diagrams (aka OODMs)** ⚡⚡⚡
- [W9.2] **Activity Diagrams** ⚡⚡⚡
- [W9.3] **Conceptualizing a Design** ⚡⚡
- [W9.4] **Architecture Diagrams: Drawing** ⚡
- [W9.5] **Design Principles** ⚡⚡
- [W9.6] **[Revisiting] SDLC Process Models** ⚡⚡
- [W9.7] **SDLC Process Models (continued)** ⚡⚡⚡
- [W9.8] **Writing Developer Documents** ⚡⚡

Admin:

1. Submit weekly quiz (P)

tP: v1.3

1. Plan the first product release (`v1.3`)
2. Manage the iteration, and deliver `v1.3`

Thu, Mar 20th 23:59

Week 9

Topics:

- [W9.1] **Conceptual Class Diagrams (aka OODMs)** ⚡⚡⚡
- [W9.2] **Activity Diagrams** ⚡⚡⚡
- [W9.3] **Conceptualizing a Design** ⚡⚡
- [W9.4] **Architecture Diagrams: Drawing** ⚡
- [W9.5] **Design Principles** ⚡⚡
- [W9.6] **[Revisiting] SDLC Process Models** ⚡⚡
- [W9.7] **SDLC Process Models (continued)** ⚡⚡⚡
- [W9.8] **Writing Developer Documents** ⚡⚡

Admin:

1. Submit weekly quiz Ⓟ

tP: v1.3

1. Plan the first product release (`v1.3`)
2. Manage the iteration, and deliver `v1.3`

Thu, Mar 20th 23:59

Aim to create a working MVP by this deadline

Thank you!

Dr Ganesh Neelakanta Iyer

gni@nus.edu.sg
http://ganeshniyer.github.io

https://www.linkedin.com/in/ganeshniyer/